

AI-POWERED CONTRACT INTELLIGENCE & RISK SCORING SYSTEM

Project Documentation:

Organization: Zaalima Development Pvt. Ltd.

Project Domain: Data Science and Machine Learning (DS&ML)

Technical Areas: Artificial Intelligence, Natural Language Processing, Transformer-Based Models, Named Entity Recognition, and Risk Scoring

Project Type: Remote Internship Project

Submitted by: Vansh Rahul Mehta

Department: Computer Engineering

Institution: Universal College of Engineering, Kaman, Bhiwandi

Academic Year: 2026–2027

Internship Duration: 25 Aug 2026 – 25 Sept 2026

Report Submission Date: 9 October 2026

DECLARATION

I hereby declare that the project entitled “AI-Powered Contract Intelligence & Risk Scoring System” has been developed as part of my internship work at Zaalima Development Pvt. Ltd.

This report describes the system architecture, implementation methodology, technologies used, risk assessment process, testing procedures, and results obtained during the development of the project.

The information presented in this report is intended to document the implemented system and its observed behavior. Any limitations of the system are acknowledged, and the output is intended to support human review rather than replace professional legal judgment.

Name: Vansh Rahul Mehta

Signature: *Vansh Mehta*

Date: 9th Oct 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Zaalima Development Pvt. Ltd. for providing me with the opportunity to work on the AI-Powered Contract Intelligence & Risk Scoring System.

This project provided valuable practical experience in developing an end-to-end Artificial Intelligence application involving document processing, Natural Language Processing, transformer-based models, named entity recognition, vector embeddings, backend API development, and risk assessment.

I would also like to thank my mentors, faculty members, and everyone who provided guidance and support throughout the development of this project.

The project helped me understand how machine learning and NLP techniques can be integrated into a practical software system to solve document analysis problems and assist users in identifying potentially important contractual provisions.

ABSTRACT

Contracts are essential documents in business, employment, procurement, and legal operations. However, reviewing lengthy contractual documents manually can be time-consuming and may make it difficult to identify unusual clauses, unfavorable obligations, missing protections, and potentially risky contractual language.

The AI-Powered Contract Intelligence & Risk Scoring System is designed to assist in the analysis of contractual documents using Artificial Intelligence and Natural Language Processing techniques. The system processes uploaded contract documents, extracts their textual content, identifies individual clauses, classifies contractual text, extracts named entities, identifies contractual indicators, and assigns risk scores to clauses.

The system incorporates an OCR-based document-processing component for extracting text from supported PDF documents, a clause extraction module for identifying contractual provisions, a Legal-RoBERTa-based classification component for analyzing clause text, and a spaCy-based Named Entity Recognition component for extracting relevant entities. Sentence-transformer embeddings are used to represent clause text numerically for semantic similarity and vector-based retrieval capabilities.

A risk-scoring module combines a baseline score, clause-type weights, contractual indicator points, and review-signal adjustments to generate clause-level risk assessments. These assessments are aggregated into an overall contract risk summary. Clauses requiring additional attention are highlighted for human review.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System
6. Proposed System
7. System Requirements
8. Technologies Used
9. System Architecture
10. Methodology
11. Risk Scoring Methodology
12. System Implementation
13. API Design
14. Testing and Validation
15. Results and Discussion
16. Limitations
17. Future Enhancements
18. Conclusion
19. References
20. Appendix

1. INTRODUCTION

1.1 Background

Contractual documents contain legally significant information concerning obligations, payment terms, confidentiality, termination, liability, indemnification, intellectual property, and other business arrangements.

As the number and complexity of contracts increase, reviewing every document manually becomes difficult. Reviewers must identify relevant provisions, understand their implications, and determine which clauses require closer attention.

Natural Language Processing enables computer systems to process and analyze human language. Transformer-based language models can represent contextual relationships between words and sentences, while Named Entity Recognition can identify relevant names, organizations, dates, and other structured information.

By integrating these techniques into a document-analysis pipeline, it is possible to automate parts of the initial contract-review process.

1.2 Project Overview

The AI-Powered Contract Intelligence & Risk Scoring System is an application that analyses contractual documents and generates structured information about their contents.

The system accepts supported contract documents, extracts text, divides the document into clauses, analyses each clause using NLP components, extracts entities and contractual indicators, and assigns risk levels.

The results are presented through an API-driven application interface and include clause-level assessments, an overall risk summary, and recommendations for human review.

The application is intended to assist contract reviewers by highlighting provisions that may deserve further investigation.

2. PROBLEM STATEMENT

Manual contract review requires significant time and attention, particularly when documents contain numerous clauses and complex legal language.

Traditional document-reading approaches do not automatically provide structured clause classification, entity extraction, risk scoring, or prioritized review recommendations. As a result, potentially important contractual provisions may be difficult to locate and compare consistently across documents.

The problem addressed by this project is the development of an AI-assisted system that can process contractual documents, extract and analyse individual clauses, identify relevant contractual information, and generate interpretable risk assessments to support human review.

3. OBJECTIVES

The primary objectives of the project are:

- 1.To develop a pipeline for processing contractual documents in PDF format.
- 2.To extract text from digital PDFs and supported scanned documents using OCR where required.
- 3.To identify and separate contractual clauses for independent analysis.
- 4.To apply transformer-based NLP classification to contractual text.
- 5.To extract relevant entities using Named Entity Recognition.
- 6.To identify contractual indicators associated with potentially significant or risky provisions.
- 7.To calculate clause-level risk scores and assign risk categories.
- 8.To aggregate clause assessments into a contract-level risk summary.
- 9.To identify clauses requiring additional human attention.

4. SCOPE OF THE PROJECT

The scope of the project includes the following functionalities:

- Contract document ingestion and text extraction.
- OCR-assisted processing for supported scanned documents.
- Clause segmentation and preprocessing.
- Transformer-based clause classification.
- Named Entity Recognition.
- Contractual indicator extraction.
- Sentence embedding generation and vector-based semantic retrieval capabilities.
- Clause-level risk scoring.
- Contract-level risk aggregation.
- Human-review recommendations.
- REST API-based communication between the frontend and backend.
- Display of analysis results through the application interface.

The system is designed primarily for preliminary contract analysis. It does not independently determine the legal validity of a contract, guarantee the identification of every risky clause, or replace a qualified legal professional.

5. EXISTING SYSTEM

Traditional contract-review processes commonly rely on manual reading, keyword searches, document templates, and reviewer experience.

Although these approaches can be effective, they have several limitations:

- Reviewing lengthy documents requires considerable time.
- Important clauses can be difficult to locate in large documents.
- Similar provisions may be interpreted inconsistently by different reviewers.
- Extracting entities and contractual information manually can be repetitive.
- Prioritizing documents according to potential risk can be challenging.
- Manual processes may not scale efficiently as document volume increases.

Some document-management tools provide keyword search and text extraction, but these features alone do not necessarily provide contextual clause classification, entity extraction, and an integrated risk summary.

6. PROPOSED SYSTEM

The proposed system integrates document processing, NLP, and risk assessment into a single workflow.

When a user submits a contract, the system extracts its text and divides it into clauses. Each clause is passed through the relevant analysis components to obtain classification results, entities, and contractual indicators.

The risk-scoring module combines these signals to generate a score and risk category for each clause. The clause-level results are then aggregated into a contract-level summary.

The frontend presents the results so that users can identify significant clauses and focus their attention on provisions recommended for human review.

Key Features:

Document processing: Extracts text from supported PDF documents, using OCR for scanned pages where required.

Clause extraction: Divides the extracted document text into smaller contractual units for independent analysis.

AI-based classification: Uses a Legal-RoBERTa-based classifier to analyze contractual language.

Entity extraction: Uses spaCy-based Named Entity Recognition to identify relevant entities in clause text.

Contractual indicator detection: Identifies configured contractual signals that contribute to the review process.

Risk scoring: Assigns numeric scores and LOW, MEDIUM, or HIGH risk categories to clauses.

Contract-level aggregation: Combines clause-level results into an overall contract risk summary.

Human-review recommendations: Highlights clauses requiring closer examination.

API integration: Provides contract-analysis functionality through FastAPI endpoints.

Frontend visualization: Presents the results in a structured interface.

7. SYSTEM REQUIREMENTS

7.1 Hardware Requirements

The following represents a general development environment suitable for the application. Actual requirements depend on document size and the resources needed to load the NLP models.

- Processor: Modern multi-core CPU.
- RAM: 8 GB or more recommended.
- Storage: Sufficient space for source code, datasets, model files, and processed documents.
- Operating system: Windows, Linux, or macOS.
- GPU: Optional, depending on the model-inference configuration.

7.2 Software Requirements

- Python 3.x
- FastAPI
- Uvicorn
- PyTorch
- Hugging Face Transformers
- spaCy.
- Sentence Transformers
- PyMuPDF
- Pytesseract or Tesseract OCR
- NumPy and other required Python dependencies
- npm
- Git
- .venv

The exact installed versions should be documented from the project's dependency files or environment.

8. TECHNOLOGIES USED

8.1 Python

Python is used for backend development, document processing, NLP integration, risk scoring, and pipeline orchestration. Its ecosystem provides libraries for machine learning, text processing, and API development.

8.2 PyTorch

PyTorch provides the deep-learning framework required to load and execute compatible transformer models.

8.3 Hugging Face Transformers

The Transformers library supports loading pretrained transformer architectures, tokenizers, and classification models for text analysis.

8.4 Legal-RoBERTa

Legal-RoBERTa is used as the transformer-based classification component for contractual text. The implementation uses the configured Legal-RoBERTa-based model and tokenizer to generate classification outputs.

8.5 spaCy

spaCy is used for Natural Language Processing tasks, including Named Entity Recognition. Extracted entities provide structured information that can help reviewers understand the parties, references, and other information appearing in clauses.

8.6 Sentence Transformers

The sentence-transformers model is used to generate 384-dimensional embeddings for clause text.

These embeddings provide numerical representations that can support semantic similarity calculations and vector-based retrieval.

8.7 PyMuPDF

PyMuPDF is used to work with PDF documents, including rendering pages for OCR processing and supporting document-text extraction workflows.

8.8 Tesseract OCR or pytesseract

Tesseract OCR, accessed through pytesseract, is used to recognize text in supported scanned document pages.

OCR quality depends on the scan resolution, page layout, and clarity of the original document.

8.9 FastAPI

FastAPI provides the backend API through which the frontend submits documents and retrieves contract-analysis results.

8.10 Frontend Technologies

The frontend provides the user interface for document submission and result presentation. It communicates with the backend using HTTP requests and displays the returned analysis information.

The exact frontend framework and libraries should be recorded from the project's package configuration.

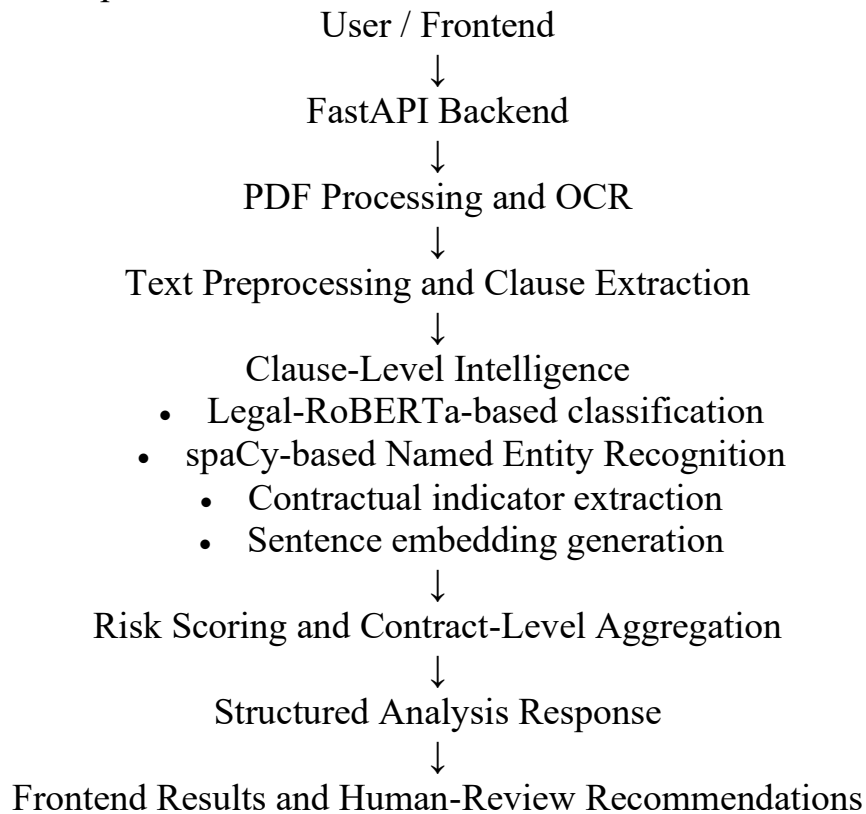
8.11 Git

Git is used to manage source-code changes and maintain the project's development history.

9. SYSTEM ARCHITECTURE

9.1 Architecture Overview

The workflow is represented below:



10.METHODOLOGY

The implementation follows a modular document-analysis pipeline.

10.1 Document Ingestion

The process begins when the user submits a supported contract document through the application interface.

The backend receives the document and passes it to the document-processing component.

Input validation and appropriate error handling are important to prevent invalid or unsupported files from being processed.

10.2 Text Extraction and OCR

The document-processing component extracts text from the PDF.

The resulting text is passed to the preprocessing component.

OCR output may contain formatting errors, incorrectly recognized characters, or incomplete text. Therefore, the extracted content should be reviewed when document quality is poor.

10.3 Text Preprocessing

The extracted text is cleaned and normalized to improve downstream processing.

Preprocessing may include:

- Removing unnecessary whitespace.
- Normalizing line breaks.
- Preparing the text for clause segmentation.

The preprocessing component aims to preserve the meaning of contractual provisions while reducing irrelevant document noise.

10.4 Clause Extraction

Contracts generally contain multiple provisions addressing different subjects.

The clause-processing component divides the extracted text into smaller units so that each unit can be analysed independently.

The extraction process uses the configured clause-segmentation logic and document-structure signals.

Each extracted clause is passed to the downstream intelligence components.

Accurate clause extraction is important because incorrectly split or merged text can affect classification and risk scoring.

10.5 Clause Classification

The extracted clause text is processed using the configured Legal-RoBERTa-based classification model.

The text is tokenized and supplied to the model, which generates classification outputs according to its trained label configuration.

The classification output contributes to the clause's structured assessment and subsequent risk analysis.

The quality of the output depends on the model's training data, label definitions, and similarity between the input contracts and the data used to train or fine-tune the model.

10.6 Named Entity Recognition

The system uses spaCy-based Named Entity Recognition to extract relevant entities from clause text.

Depending on the configured NLP pipeline, extracted information can include names, organizations, dates, and other recognized entity types.

The extracted entities provide additional context for understanding contractual provisions.

Entity extraction is not guaranteed to identify every legally significant entity, particularly when the source text contains OCR errors or unusual formatting.

10.7 Contractual Indicator Extraction

In addition to classification and entity extraction, the system identifies configured contractual indicators.

These indicators act as additional signals during the risk-assessment process. Depending on the implemented rules, they may represent language or conditions that deserve additional review.

The indicator results are combined with clause-type information and review signals to support the risk-scoring component.

10.8 Sentence Embedding Generation

The system uses sentence-transformers to generate vector representations of clause text.

Each embedding has 384 dimensions.

These representations support semantic similarity and vector-based retrieval functionality. They allow text to be represented numerically so that related clauses can be compared using suitable similarity measures.

Embeddings are supplementary to the classification and risk-scoring workflow; an embedding alone does not determine whether a clause is legally risky.

10.9 Risk Assessment

Each clause is evaluated using the implemented risk-scoring logic.

The scoring component combines a baseline score, clause-type weights, contractual indicator points, and review-signal adjustments.

The resulting score is bounded to the configured range and mapped to a risk category.

10.10 Contract-Level Aggregation

After clause-level assessments are generated, the system aggregates the results to produce a contract-level risk summary.

This design aims to represent both the overall distribution of clause risk and the presence of particularly concerning clauses.

10.11 Human-Review Recommendations

The system identifies clauses that require additional attention based on its configured review signals and risk-assessment logic.

The recommendations help users prioritize manual examination.

The system supports decision-making by highlighting potentially important information; it does not make the final legal decision.

11. RISK SCORING METHODOLOGY

11.1 Purpose

The risk-scoring component converts clause-analysis signals into a numeric score and a corresponding risk category.

This makes it easier to prioritize clauses and summarize the results of an entire contract.

11.2 Clause-Level Scoring

The implemented scoring logic uses the following conceptual structure:

Clause risk score = Baseline score + Clause-type contribution + Contractual indicator points + Review-signal adjustment

The final score is capped at 100.

The baseline score provides a starting value. Clause-type weights contribute according to the configured classification or clause-type rules. Contractual indicators and review signals modify the score based on the implemented scoring configuration.

The exact numerical contribution of each clause type, indicator, and review signal is defined in the project's risk-scoring implementation.

11.3 Risk Categories

The implemented score ranges are:

Risk category	Score range	Interpretation
---------------	-------------	----------------

LOW	0–39	Lower-priority clauses under the configured scoring rules
MEDIUM	40–69	Clauses requiring additional attention
HIGH	70–100	Clauses prioritized for closer review

These categories represent the application's configured assessment levels. They do not constitute a formal legal opinion.

11.4 Contract-Level Risk Score

The contract-level score is calculated using a severity-weighted average of clause assessments, together with a capped premium for high-risk clauses.

This approach allows the summary to account for the distribution of risk across clauses while recognizing the significance of high-risk provisions.

The exact aggregation weights and premium cap are defined by the implemented scoring module.

11.5 Interpretation

The score should be interpreted alongside the underlying clause text, classification output, extracted entities, contractual indicators, and review recommendations.

A numeric score provides a prioritization mechanism, not proof that a clause is legally enforceable, unenforceable, acceptable, or unacceptable.

12. SYSTEM IMPLEMENTATION

The project follows a modular software architecture that separates document processing, NLP analysis, risk assessment, and API functionality.

- 1 Document Processing Module
- 2 Clause Processing Module
- 3 Contract Intelligence Module
- 4 Risk Scoring Module
- 5 Pipeline Orchestration
- 6 Backend API
- 7 Frontend Integration

13. API DESIGN

The backend exposes the following principal endpoints.

Method	Endpoint	Purpose
GET	/	Root endpoint
GET	/health	Health-check endpoint
POST	/analyse	Contract-analysis endpoint

The application also exposes the OpenAPI specification and interactive API documentation through the standard FastAPI documentation routes.

13.1 Health Check

Endpoint: GET /health

The health-check endpoint is used to verify that the backend application is responding.

The exact response format is determined by the implemented endpoint.

13.2 Contract Analysis

Endpoint: POST /analyze

The analysis endpoint receives a contract document according to the API's configured request format and initiates the document-analysis pipeline.

The response contains the generated analysis results, including the risk-scoring and

risk-summary information.

The detailed request schema, response fields, and validation rules should be verified against the running application's OpenAPI specification.

13.3 API Documentation

FastAPI provides interactive API documentation at: /docs

When running locally with the default Uvicorn configuration, this is typically available at: <http://127.0.0.1:8000/docs>

The interactive documentation can be used to inspect the available routes and test supported requests.

15. RESULTS AND DISCUSSION

The project successfully integrates document processing, clause extraction, NLP-based analysis, risk scoring, and API delivery into a contract-intelligence workflow.

The recorded pipeline execution produced 37 extracted clauses and 37 risk assessments. Of these clauses, 14 were identified as requiring human review, 18 contained configured contractual indicators, and 34 had extracted entities.

The results demonstrate the ability of the system to generate structured information from a contract and identify provisions for closer examination.

The risk-scoring component provides a consistent framework for organizing clause assessments into LOW, MEDIUM, and HIGH categories. The contract-level aggregation produces a consolidated summary that can help users prioritize their review.

The integration of transformer-based classification, entity extraction, and contractual indicators provides multiple sources of information for the assessment process.\

The practical usefulness of the results depends on document quality, clause-

extraction accuracy, the model's training and configuration, the quality of the indicator rules, and appropriate human interpretation.

The current results should therefore be considered evidence of pipeline execution rather than a claim of independently measured predictive accuracy.

Figure 1: Application Interface

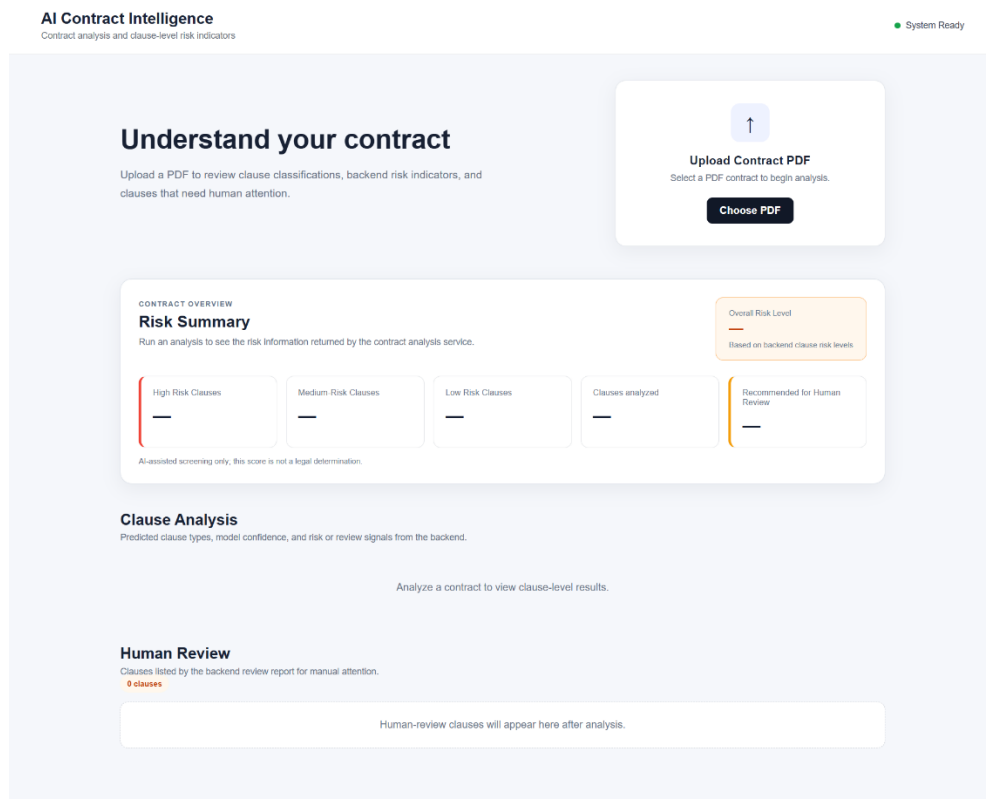


Figure 2: Contract Analysis Results

```
(.venv) PS C:\Users\Vansh Mehta\OneDrive\Desktop\internship-project-1> python -m uvicorn main:app --reload

End-to-end pipeline completed successfully.

Clauses detected: 190
Clauses processed: 190
Processing errors: 0
Low-confidence clauses: 98

Clause-level output:
data\processed\contract_clause_intelligence.json

Contract-level output:
data\processed\contract_intelligence_report.json

Human-review output:
data\processed\contract_review_report.json

Vector index:
data\vector_store\contract_clauses.index

Vector metadata:
data\vector_store\contract_clauses.json
INFO: 127.0.0.1:57179 - "POST /analyze HTTP/1.1" 200 OK
INFO: 127.0.0.1:55557 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:56834 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:55557 - "GET /openapi.json HTTP/1.1" 200 OK
```

Figure 3: Clause-Level Risk Assessment

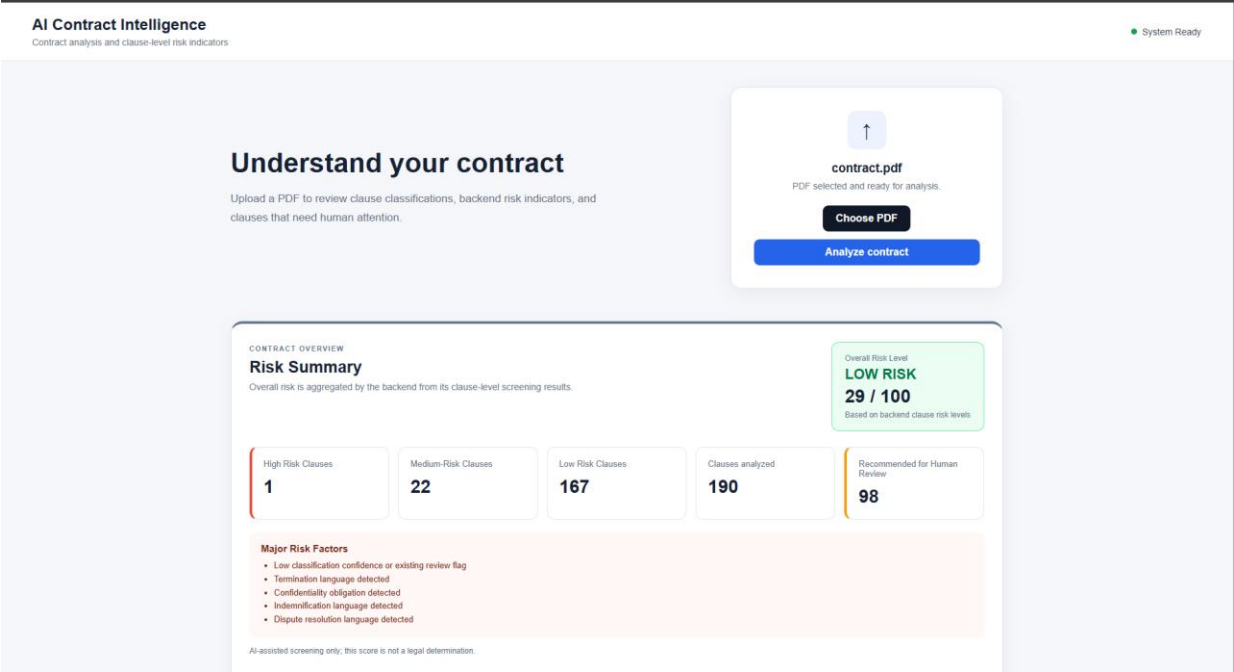


Figure 4: API Documentation

AI Contract Intelligence API 1.0.0 OAS 3.1

/openapi.json

API for contract analysis and intelligence extraction.

default

GET	/	Root	⌵
GET	/health	Health Check	⌵
POST	/analyze	Analyze Contract	⌵

Schemas

Body_analyze_contract_analyze_post > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

16. LIMITATIONS

The following limitations should be considered when interpreting the system's output.

OCR limitations: Scanned documents with poor image quality, unusual fonts, etc may produce inaccurate text.

Clause extraction limitations: Unusual numbering, tables, or inconsistent formatting may affect segmentation.

Model limitations: Classification quality depends on the model configuration, training data, and the relationship between training examples and real-world contracts.

Entity extraction limitations: General-purpose entity recognition may not capture every legal entity, contractual reference, or domain-specific term.

Rule-based indicator limitations: Configured indicators may not capture every

possible contractual risk and may flag language that requires contextual interpretation.

Legal limitations: The system does not replace qualified legal counsel or independently determine legal enforceability.

17. FUTURE ENHANCEMENTS

The following enhancements could improve the system in future versions:

1. **Expanded contract coverage:** Extend support for additional contract types and document formats.
2. **Improved clause extraction:** Introduce more robust document-layout analysis and handling of nested clauses.
3. **Model evaluation:** Evaluate classification performance using independently annotated contracts and metrics such as precision, recall, and F1-score.
4. **Risk-score calibration:** Compare the scoring framework against expert-reviewed examples and refine the weights.
5. **Explainable risk assessments:** Display the indicators, model outputs, and scoring contributions that influenced each assessment.

6. **Human feedback:** Allow reviewers to correct classifications and provide feedback for future model improvement.
7. **Document comparison:** Compare different versions of a contract and identify changes to important provisions.
8. **Security enhancements:** Add suitable access controls, secure document handling, retention policies, and audit logging.
9. **Performance optimization:** Improve processing efficiency for large documents and concurrent requests.
10. **Expanded reporting:** Generate downloadable reports containing clause-level results, risk summaries, and review notes.

18. CONCLUSION

The AI-Powered Contract Intelligence & Risk Scoring System demonstrates the practical application of Artificial Intelligence and Natural Language Processing to contractual document analysis.

The system combines PDF processing and OCR, clause extraction, Legal-RoBERTa-based text classification, spaCy-based Named Entity Recognition, sentence embeddings, contractual indicator detection, and risk-scoring logic.

The integrated pipeline generates clause-level risk assessments, organizes them into LOW, MEDIUM, and HIGH categories, and aggregates the results into a contract-level summary. It also identifies clauses that require closer human examination.

The recorded execution, which produced 37 clause assessments from 37 extracted clauses, demonstrates that the major components can operate together within the application workflow.

The project provided practical experience in building a modular AI application, integrating NLP models into a backend service, implementing risk-assessment logic, and connecting the processing pipeline to a frontend interface.

Overall, the system serves as a foundation for AI-assisted contract review. Its results should be interpreted as decision-support information, with final contractual judgments remaining the responsibility of qualified human reviewers.

19. REFERENCES

1. Hugging Face, Transformers Documentation. <https://huggingface.co/docs/transformers/>
2. PyTorch Documentation. <https://pytorch.org/docs/>
3. spaCy Documentation. <https://spacy.io/usage>
4. Sentence Transformers Documentation. <https://www.sbert.net/>
5. FastAPI Documentation. <https://fastapi.tiangolo.com/>
6. PyMuPDF Documentation. <https://pymupdf.readthedocs.io/>
7. Tesseract OCR Documentation. <https://tesseract-ocr.github.io/>
8. CUAD: Contract Understanding Atticus Dataset. <https://www.atticusprojectai.org/cuad>
9. Legal-RoBERTa model and classifier documentation corresponding to the model checkpoints used in the project.

20. APPENDIX

A. Backend Startup

From the project root directory, the backend can be started using the configured Python environment:

```
python -m uvicorn src.api.main:app --reload
```

The interactive API documentation is typically available at:

<http://127.0.0.1:8000/docs>

B. Principal Implementation Components

The project includes the following implementation components:

- `src/api/main.py` — FastAPI application entry point.
- `src/pipeline/contract_pipeline.py` — Contract-analysis pipeline orchestration.
- `src/data_processing/ocr_processor.py` — OCR and document-processing functionality.

- `src/data_processing/clause_processor.py` — Clause processing and extraction.
- `risk_scoring.py` — Risk-scoring functionality, located in the project's configured module.
- `test_risk_scoring.py` — Risk-scoring tests, located in the project's configured test directory.

The precise locations of the risk-scoring implementation and test files should be confirmed against the final repository structure.

C. Sample Test Artifacts

- `data/test_contracts/low_risk_contract_day23.pdf`
- `data/test_contracts/high_risk_contract_day23.pdf`

These artifacts support the documented pipeline-validation workflow.

D. Submission Contents

The final submission should include the project report, source code, required configuration examples, dependency files, relevant test files, and any additional artifacts required to reproduce the implementation.

Sensitive information, credentials, private contractual documents, and unnecessary generated files should not be included in the submission.

End of Report